How the Chipless Reader Works

A Mac app tells an STM32 what to read. The STM32 sweeps a radio tone across 860–865 MHz, looks for the notches a chipless tag cuts into the echo, turns them into a 10-bit pattern, and identifies the tag from a database it holds in flash.

Firmware 0.3 · STM32F407G-DISC1 · November 2, 2026

Working on the real board

USB link, sweep and decode, database lookup, and settings that survive a power cycle. All 61 end-to-end checks pass on the chip.

Simulated for now

The radio signal and the tags. The firmware models them until the RF chips are wired. Every result in this document uses that model.

The open question

Whether real tags can hold 10 bits in 5 MHz. It takes resonators sharper than printed copper usually gets. See What could stop it working.

How a tag holds an ID

A chipless tag has no chip and no battery. It stores its ID in its copper shape: a row of resonators, each tuned to one frequency. A resonator that is present absorbs energy at its frequency and leaves a dip, called a notch, in the echo. That is a **1**. A missing resonator leaves no dip, which is a **0**.

Ten bits in the 5 MHz band gives ten cells 500 kHz wide, with one resonator at the centre of each: 860.25, 860.75, and so on up to 864.75 MHz. Patterns are written with the lowest frequency first, so the tag below has resonators at f1, f3, f5, f6, f9 and f10.

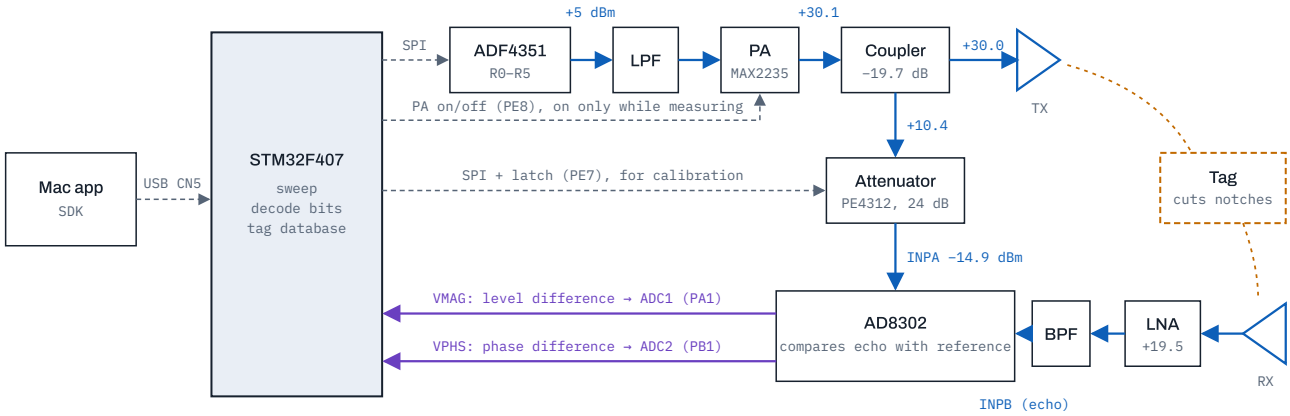
f1	f2	f3	f4	f5	f6	f7	f8	f9	f10
1	0	1	0	1	1	0	0	1	1

The pattern 1010110011, which is ID 0x335 inside the software (bit 0 is f1).

The same scheme gives the smaller formats: a single-bit tag has one resonator at 862.5 MHz, and a double-bit tag has two, at 861.25 and 863.75 MHz. Because the ID is copper, it can't be rewritten, and a tag can't tell the reader anything about itself. The reader has to know the format in advance.

The system

The Mac decides what to do. The STM32 decides how. It controls the RF chips over a few wires and reads back two voltages that describe the echo.



Levels are in dBm at the default settings, from the datasheets, not measured. Blue is the RF path, grey is control, purple is the two voltages the STM32 reads.

What happens in one read

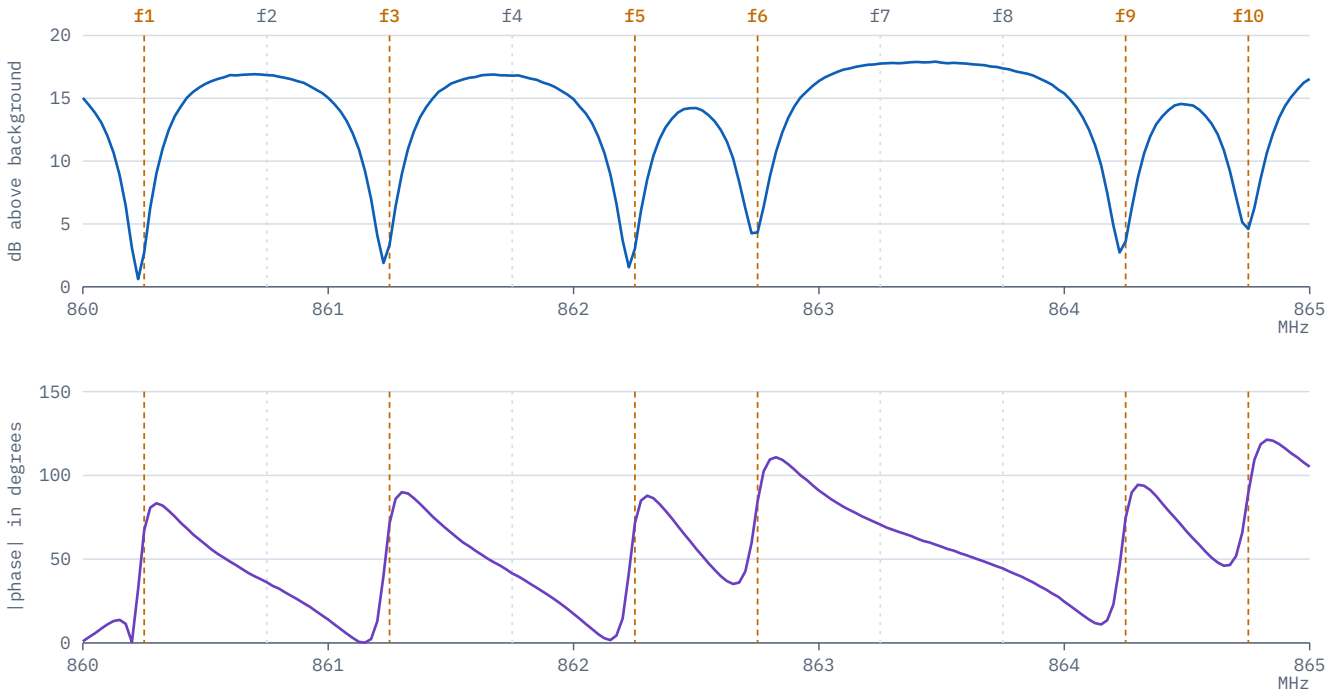
A read takes about 100 ms on the chip.

1. **The app asks.** Pressing Read Tag sends a small packet over USB. On the wire it is `A5 04 00 30 00 9A 18`: a start marker, the length, the READ_TAG command, a status byte and a checksum.
2. **The PA turns on** and the STM32 waits 1 ms for its power ramp to finish.
3. **The sweep runs across 201 frequencies,** from 860 to 865 MHz in 25 kHz steps. At each one, the STM32 programs the ADF4351, waits for the PLL to lock and then 200 µs for things to settle, and takes 8 readings of VMAG and VPHS. ADC1 and ADC2 sample together, so magnitude and phase describe the same moment.
4. **The voltages become dB and degrees.** The AD8302 gives 28.7 mV per dB and 10.1 mV per degree at this frequency.
5. **The PA turns off,** so the board isn't radiating between reads.
6. **The background is removed.** A sweep captured earlier with no tag present is subtracted, which takes out antenna leakage and room reflections.
7. **Each bit is decided** from its notch depth and phase swing, as described below.
8. **The pattern is looked up** in the tag database, and the name goes back to the app with the signal strength, margin and per-bit numbers.

In continuous mode the reader repeats this at the profile's read rate, every 250 ms by default, and sends a report each time it finds a tag.

A real read, point by point

This is the reader's own output for the tag 1010110011 at 20 cm. It comes from the firmware with its simulated radio. Each notch lines up with a marked 1, and the 0 bits sit on flat shoulders.



Amber dashed lines mark the 1 bits, grey dotted lines the 0 bits. The phase jumps sharply through every notch, which gives the reader a second, independent sign of a resonator.

BIT	MHZ	NOTCH DEPTH (DB)	PHASE SWING (°)	DECISION
f1	**860.25**	**12.3**	**83**	**1**
f2	860.75	0.0	16	0
f3	**861.25**	**11.7**	**90**	**1**
f4	861.75	0.0	17	0
f5	**862.25**	**11.2**	**86**	**1**
f6	**862.75**	**9.9**	**76**	**1**
f7	863.25	0.0	12	0
f8	863.75	0.0	12	0
f9	**864.25**	**10.9**	**83**	**1**
f10	**864.75**	**9.4**	**75**	**1**

The reader reported TEN-A, RSSI −25.2 dBm, margin 3.4 dB. Margin is how close the least certain bit came to the threshold: 9.4 dB at f10 is 3.4 dB above the 6 dB threshold.

How a bit is decided

- **Notch depth.** The reader finds the lowest level within ±100 kHz of the bit frequency, which allows for a notch sitting slightly off nominal. It then finds the highest level from there out to the midpoint with each neighbour, 250 kHz away. These are the shoulders. Depth is the lower

shoulder minus the lowest level. Using the lower shoulder stops a 0 next to a 1 from borrowing that notch's slope.

- **Phase swing.** How far the AD8302's phase moves within the same ±100 kHz.
- **The rule.** A bit is 1 when the depth is at least 6 dB and the swing at least 15°. A depth of 12 dB or more doesn't need the phase. The check exists because a level step with no resonance behind it, such as a reflection, moves no phase.

All three numbers can be changed in the app under Reader Setup. A tag with no resonators at all, 0000000000, looks the same as no tag, so that pattern can't be used.

What the reader has to be told

A chipless tag can't announce its format, so the tags and the reader must agree on it in advance. The firmware keeps that agreement in two places, both stored in flash when you press Save to Reader.

TAG PROFILE	DEFAULT
Sweep	860–865 MHz, 25 kHz steps, 201 points
Bit frequencies	10, evenly spaced. These are placeholders until real tags are measured
Speed against noise	200 µs settle, 8 samples per point
Read rate	Every 250 ms in continuous mode
Bit decision	6 dB notch, 15° phase
Database tolerance	Exact match. Can allow a set number of differing bits

The **tag database** holds up to 64 tags on the STM32, each with a name of up to 16 characters, a pattern and a format. A pattern that isn't in the list is reported as not in the database rather than given a wrong name. Because the database is on the reader, identification doesn't depend on the Mac.

Gain and attenuation

The AD8302 only gives meaningful numbers when both inputs are within −60 to 0 dBm and within 30 dB of each other. Two settings control this.

Gain

The app sets the ADF4351's output to −4, −1, +2 or +5 dBm. The MAX2235 PA is close to saturation at these levels, so the 9 dB range only moves the power at the antenna from about 27.1 to 30.0 dBm. This also confirms the note on the diagram that no preamp is needed.

Attenuation

The PE4312 sets the reference level at INPA, from 0 to 31.5 dB in 0.5 dB steps. It sits on the reference path, not the receive path, so it doesn't change how strong the echo is. The default is 24

dB, the value noted on the block diagram, which gives −14.9 dBm.

- **Safety limit.** At 0 dB the reference would be about +9 dBm, close to the AD8302's +10 dBm absolute maximum. With the PA on, the firmware refuses anything below 9.5 dB.
- **Auto-level.** The reader sweeps, finds the middle of the measured range, and picks the attenuation that centres it on the AD8302.
- **Attenuator scan.** The reader steps the attenuator and records VMAG. A straight line shows the PE4312 is wired correctly, and its slope is the real mV/dB of your AD8302.

The dBm figures rely on a chain of gains and losses taken from the datasheets. All of them can be replaced with measured values on the Calibration screen. Notch detection compares levels within a sweep, so it doesn't depend on them.

Programming a frequency

The ADF4351 runs its VCO between 2.2 and 4.4 GHz and divides down to the output. To reach f1, the STM32 works out the settings each time. No values are stored in advance.

```
A5 08 00 20 00 5A 20 0D 00 12 D6     SET_LO, 860250 kHz

VCO   860.25 MHz × 4 = 3441 MHz
N     3441 ÷ 25 MHz reference = 137 + 16/25

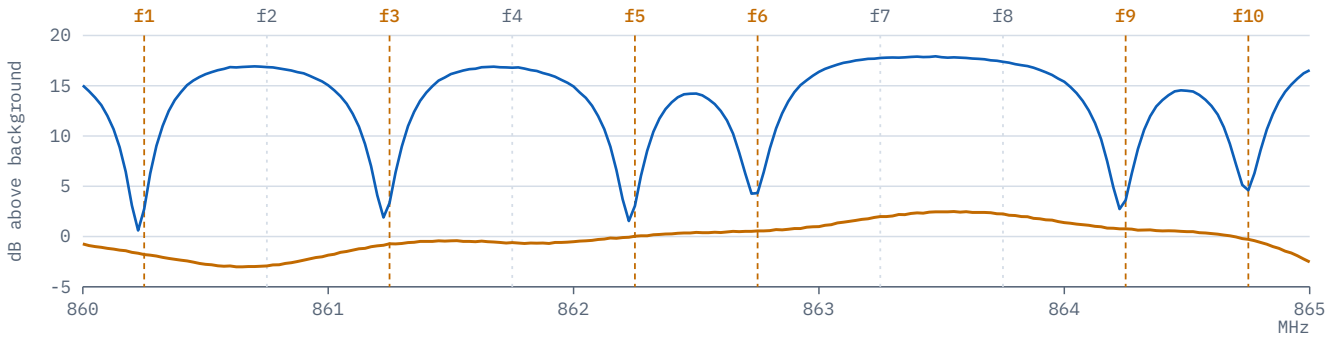
R5 00580005   R4 00AC803C   R3 000004B3
R2 18004E42   R1 080080C9   R0 00448080
```

Writing R0 last starts the chip's VCO band selection, which takes about 80 µs. During a sweep only the registers that changed are sent, which saves time on every point.

What could stop it working

Resonator sharpness

This is the biggest risk. A notch's width is its frequency divided by the resonator's Q. Bits 500 kHz apart only stay separate if each notch is narrower than that spacing. Below, the same tag is read at two values of Q.



Blue: Q 2000, read correctly as 1010110011. Amber: Q 250, where the notches overlap so much that the echo sinks to the leakage level. No separate dips remain, and the reader finds no tag. Both from the simulated radio at 20 cm.

RESONATOR Q	NOTCH WIDTH	1 BIT	2 BITS	10 BITS
100	8.62 MHz	5/5	0/5	0/25
250	3.45 MHz	5/5	5/5	0/25
500	1.73 MHz	5/5	5/5	0/25
1000	0.86 MHz	5/5	5/5	10/25
1250	**0.69 MHz**	**5/5**	**5/5**	**25/25**
2000	**0.43 MHz**	**5/5**	**5/5**	**25/25**

Ten bits need a Q of about 1250 to read at all, and about 2000 to read with margin. Printed copper resonators usually reach a few hundred. If real tags come in there, the choices are to widen the band or use fewer bits. Both are settings in the app and need no firmware change. These numbers come from a model that assumes a 20 dB notch, so **the deciding step is measuring a real tag**, on a VNA or in a CST simulation with realistic losses.

Antenna isolation

With about 1 W at the TX antenna, leakage into the RX antenna reaches the AD8302 after 18 dB of net gain. The antennas need at least **48 dB** of isolation to stay inside the AD8302's range, and at least **38 dB** to stay under its absolute maximum. Measure this before connecting INPB, or fit a 20 dB pad in front of it for the first power-up.

Manufacturing tolerance

The reader searches ±100 kHz around each bit. Printed tags can shift by much more than that from board to board, which is why bit frequencies are editable and should come from measurements.

One tag at a time

Chipless tags can't take turns. Two tags in front of the antennas mix their notches, so the reader identifies one tag at a time from a known set.

Licensing

In India the licence-exempt RFID band starts at 865 MHz, so most of 860–865 MHz needs authorisation to radiate. Until that is settled, test in a shielded enclosure or with cabled connections.

What is real and what is simulated

STEP	STATUS
USB link, frames, checksums, bad-data recovery	Tested on the board
ADF4351 settings for any frequency	Tested on the board (values computed and checked; no chip attached)
Sweep loop and its timing	Tested on the board: 201 points in 100 ms
Notch and phase measurement, bit decision, database lookup	Tested on the board, on simulated readings
Saving settings and the database to flash	Tested on the board, through a power cycle
The AD8302 readings at each point	**Simulated** by `firmware/core/rf_sim.c`
SPI signals reaching real chips, PLL lock, ADC1 + ADC2 sampling	**Not tested.** Needs the RF parts and the hardware firmware build
Real tags, real echo levels	**Not tested**

The simulator is a physics model, not a list of answers. It reads the ADF4351 settings the driver just sent, works out the frequency the way the chip would, and models the chain and tag. The decoder only ever sees the 201 readings. Switching to real hardware means flashing the other build, `prebuilt/hw/chipless_reader.elf`. The app and everything above that layer stay the same.

Where things are

IN THE APP	WHAT IT'S FOR
Read Tags	Read once or continuously; the result, per-bit numbers and sweep plots
Reader Setup	Tag format, bit frequencies, sweep, gain, read rate, bit decision
Tag Database	Add, edit, delete, import and export tags
Calibration	Attenuator, AD8302 levels, Auto-level, scan, RF chain values, single tone
Log	What happened, optionally every frame in hex

IN THE CODE	WHAT'S IN IT
`firmware/core/reader.c`	Sweep, notch and phase measurement, bit decision, Auto-level. Start here
`firmware/core/tagdb.c`	Tag database
`firmware/core/calibration.c`	RF chain levels, PA curve, attenuator limit
`firmware/core/adf4351.c`	Register calculation and programming
`firmware/core/commands.c`	One handler per command
`firmware/stm32/Inc/board.h`	Pin map and module settings
`docs/DESIGN_NOTES.md`	Datasheet findings, power budget, bring-up steps
`docs/PROTOCOL.md`	Every command, byte by byte

Terms

Notch	The dip a resonator cuts into the echo at its frequency
Q	How sharp a resonator is. Frequency divided by notch width
Sweep	Stepping the tone across a range of frequencies and measuring at each
Background	A sweep with no tag present, subtracted from later reads
Margin	How far the least certain bit was from the decision threshold
RSSI	Received signal strength at the RX antenna, in dBm
INPA / INPB	The AD8302's two inputs: reference and echo
VMAG / VPHS	The AD8302's outputs: voltages for level difference and phase difference
PLL, locked	The circuit holding the ADF4351 on frequency; locked means it has settled
VNA	Vector Network Analyzer: a lab instrument that shows a tag's notches directly